

PeptideIdentifier + CleavageSite predictor data preparation

Raúl Fernández-Díaz

2026-04-16

Introduction

The idea is to develop two complementary models that can be used to predict peptides within a protein. Both of the models make predictions at the residue-level.

1. **PeptideIdentifier**: It predicts which residues belong to a PEPTIDE according to the UniProtKB definition.
2. **CleavageSitePredictor**: It predicts which residues are a **start** or **end** of a PEPTIDE.

Materials and methods

Data collection

Data was gathered on 26/03/2026 from the UniProtKB with the following query:

https://www.uniprot.org/uniprotkb?query=*%26amp%3Bfacets=reviewed%3Atrue%2Cproteins_with%3A41

which looks for:

- all proteins
- that have been “Reviewed” (Swiss-Prot)
- with a “Peptide”

Data preprocessing

```

from typing import List

import numpy as np
import pandas as pd

def process_peptide(sentence: str) -> str:
    """
    Process Peptide string and output start and end points
    for a given peptide
    """
    frags = sentence.split(";")
    frags = [f.replace("PEPTIDE ", "").replace("..", "-")
              for f in frags if "PEPTIDE" in f]
    return ";".join(frags)

def peptide_mask(sites: List[int], sequence: str) -> List[int]:
    """
    Guided by the indices in the list of peptides create a mask
    where there is a "1" for residues with a cleavage point and a 0
    for all the rest
    """
    mask = np.zeros(len(sequence))
    for s in sites:
        mask[s[0]:s[1]] = 1
    return mask

```

Cleavage sites

Some entries have the indication “<1”, we have assumed them to be starting point 1. Some entries have the symbol “?” before any of the cleavage points, we have removed the whole protein. Entries with the symbol “>” we have kept, but ignored the cleavage point so indicated. Some entries are marked with “->” instead of the separator “-”, we have treated them as if they had the standard separator (“-”).

We also remove proteins with length greater than 1,024

```

def cleavage_mask(sites: List[int], sequence: str) -> List[int]:
    """
    Guided by the indices in the list of cleavage points create a mask
    where there is a "1" for residues with a cleavage point and a 0

```

```

for all the rest
"""
mask = np.zeros(len(sequence))
for s in sites:
    mask[s-1] = 1
return mask

SEQUENCE_ORDER = 0

def origin_mask(sequence: str) -> List[int]:
    global SEQUENCE_ORDER
    mask = np.ones(len(sequence))
    mask *= SEQUENCE_ORDER
    SEQUENCE_ORDER += 1
    return mask

def unravel(array: List[np.ndarray]) -> np.ndarray:
    new_array = []
    for array in array:
        for row in array:
            new_array.append(row)
    new_array = np.stack(new_array)
    return new_array

# Full preprocessing pipeline
def run():
    df = pd.read_csv("data/swissprot-peptide.tsv", sep='\t')
    df = df[['Entry', "Organism", "Length", "Peptide", "Sequence"]]
    df["Peptide"] = df["Peptide"].map(process_peptide)
    df.to_csv("processed-data/peptide.tsv", sep='\t', index=False)
    df['Sites'] = df['Peptide'].map(lambda x: x.split(";"))
    sites, peptides = [], []
    for peptide, length in zip(df['Sites'], df['Length']):
        sites.append([])
        peptides.append([])
        for site in peptide:
            if '>' in site or '<' in site or "?" in site:
                continue
            else:

```

```

        start = int(site.split('-')[0])
        end = int(site.split('-')[1])
        if start != 1:
            sites[-1].append(start - 1)
        if end != length:
            sites[-1].append(end + 1)
        peptides[-1].append((start, end))

df['Peptides'], df['Sites'] = peptides, sites
print(f"Num of proteins without sites: {len(df[df['Sites'].map(len) < 1]):,}")
df = df[df['Sites'].map(len) > 0]
long_seqs = len(df[df['Length'] > 1022])
print(f"There are {long_seqs:,} sequences that are too long")
df = df[df['Sequence'].map(len) <= 1022]
df.to_parquet("processed-data/peptide.pqt", index=False)

site_masks = df.apply(
    lambda x:
        cleavage_mask(x['Sites'], x['Sequence']),
    axis=1
)
peptide_masks = df.apply(
    lambda x:
        peptide_mask(x['Peptides'], x['Sequence']),
    axis=1
)
re = RepEngineLM('esm2-8m', average_pooling=False)
re.move_to_device("mps")
reps = re.compute_reps(
    df['Sequence'], batch_size=64
)
new_origins = unravel(df['Sequence'].map(origin_mask))
new_site_masks = unravel(site_masks)
new_peptide_masks = unravel(peptide_masks)
new_reps = unravel(reps)
df = pd.DataFrame()
df['x'] = [r for r in new_reps]
df['original-prot'] = [o for o in new_origins]
df['y-sites'] = [m for m in new_site_masks]
df['y-peptides'] = [m for m in new_peptide_masks]
df.to_parquet("processed-data/peptide.pqt")

```

Model definitions

Both PeptideIdentifier and CleavageSitePredictor models are built with the same approach:

1. Per-residue representations, i.e. vectors with (1, 320) dimension, are computed using the ESM2 8M model, for all proteins in the dataset.
2. A multi-layer perceptron (MLP), i.e. a neural network, it's trained to classify those residues as either positive (belongs-to-peptide, is-a-cleavage-site) or negative. The MLP has two hidden layers with 320 as the hidden dimension. The loss function used is a Focal loss to better handle the imbalance in the data.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class FocalLoss(nn.Module):
    def __init__(self, alpha=1.0, gamma=2.0, reduction='mean'):
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.reduction = reduction

    def forward(self, inputs, targets):
        # inputs: (batch, num_classes)
        # targets: (batch,)
        ce_loss = F.cross_entropy(inputs, targets, reduction='none')
        pt = torch.exp(-ce_loss) # probability of correct class
        focal_loss = self.alpha[targets] * (1 - pt) ** self.gamma * ce_loss

        if self.reduction == 'mean':
            return focal_loss.mean()
        elif self.reduction == 'sum':
            return focal_loss.sum()
        else:
            return focal_loss

class MLP(nn.Module):
    def __init__(self, in_features, hidden_state, layers, num_classes):
        super(MLP, self).__init__()

        self.fc1 = nn.Linear(in_features, hidden_state) # adjust based on input length
```

```

        self.layers = nn.ModuleList([nn.Linear(hidden_state, hidden_state)
                                      for n in range(layers)])
        self.final = nn.Linear(hidden_state, num_classes)
        self.final_af = nn.Softmax(-1)

    def forward(self, x):
        # x: (batch, channels, length)
        x = F.relu(self.fc1(x))
        for layer in self.layers:
            x = F.relu(layer(x))
        x = self.final_af(self.final(x))

        return x

```

Results

PeptideIdentifier results

The model was trained with the 5x5-fold cross-validation strategy for evaluation with train-test partitions created with CCPart and MMSeqs2 (Smith-Waterman) with prefiltering step and maximum similarity (60%). The model had the following hyperparameters:

```

import yaml

hparams = yaml.safe_load(open("../results/peptides-2.yml"))
for key, value in hparams.items():
    print(f"{key} - {value}")

```

```

downsampling - no
hidden_state - 320
hpo - no
layers - 2
model - nn
alpha - [0.25, 0.75]
gamma - 2.0

```

```

import pandas as pd

results_df = pd.read_csv("../results/results-peptides-predictor-2.csv")

```

```

# Select numeric columns
numeric_cols = results_df.select_dtypes(include='number').columns

# Compute mean row
mean_row = results_df[numeric_cols].mean().to_frame().T

# Label non-numeric columns
for col in results_df.columns:
    if col not in numeric_cols:
        mean_row[col] = "mean"

# Append
mean_row['seed'] = 'mean'
mean_row['fold'] = 'mean'
results_df = pd.concat([results_df, mean_row], ignore_index=True)
results_df = results_df[~results_df['mcc'].isna()]
results_df = results_df[['seed', 'fold', 'mcc', 'recall_weighted', 'precision_weighted']]
# Highlight the last row (mean row)
def highlight_last_row(row):
    if row.name == results_df.index[-1]:
        return ['background-color: yellow'] * len(row)
    else:
        return [''] * len(row)

results_df.style.apply(highlight_last_row, axis=1)

results_df.iloc[-1, :]

```

```

seed                mean
fold                mean
mcc                 0.786212
recall_weighted     0.920062
precision_weighted  0.920167
Name: 275, dtype: object

```

CleavageSitePredictor results

The model was trained with the 5x5-fold cross-validation strategy for evaluation with train-test partitions created with CCPart and MMSeqs2 (Smith-Waterman) with prefiltering step and maximum similarity (60%). con The model had the following hyperparameters:

```
import yaml
```

```
hparams = yaml.safe_load(open("../results/sites-3.yml"))  
for key, value in hparams.items():  
    print(f"{key} - {value}")
```

```
alpha - [0.1, 0.9]  
downsampling - no  
exp-number - 3  
gamma - 2.0  
hidden_state - 320  
hpo - no  
layers - 2  
model - nn
```

```
import pandas as pd
```

```
results_df = pd.read_csv("../results/results-sites-predictor-3.csv")
```

```
# Select numeric columns
```

```
numeric_cols = results_df.select_dtypes(include='number').columns
```

```
# Compute mean row
```

```
mean_row = results_df[numeric_cols].mean().to_frame().T
```

```
# Label non-numeric columns
```

```
for col in results_df.columns:  
    if col not in numeric_cols:  
        mean_row[col] = "mean"
```

```
# Append
```

```
mean_row['seed'] = 'mean'
```

```
mean_row['fold'] = 'mean'
```

```
results_df = pd.concat([results_df, mean_row], ignore_index=True)
```

```
results_df = results_df[~results_df['mcc'].isna()]
```

```
results_df = results_df[['seed', 'fold', 'mcc', 'recall_weighted', 'precision_weighted']]
```

```
# Highlight the last row (mean row)
```

```
def highlight_last_row(row):
```

```
    if row.name == results_df.index[-1]:
```

```
        return ['background-color: yellow'] * len(row)
```

```
    else:
```

```
        return [''] * len(row)

results_df.style.apply(highlight_last_row, axis=1)

results_df.iloc[-1, :]
```

```
seed                mean
fold                mean
mcc                 0.554726
recall_weighted     0.977218
precision_weighted  0.982651
Name: 275, dtype: object
```